

PacketADE Workspace, Agents, and Settings

Decision report | 29 July 2026

RECOMMENDATION

Make Workspaces CLI-first. Keep GUI agents, but give them an agent-first surface. Build that surface in the main window before making it detachable.

Workspace PacketCode + CLI command center	Agents Delegation, approvals, and review	Flight Deck Structured delivery
--	---	------------------------------------

Prepared from a three-way source audit: PacketADE implementation, current first-party competitor documentation, and the complete Settings control/persistence/runtime path. Recommendations are separated from facts.

Executive answer

Do not remove agents from PacketADE. Do stop making a GUI agent conversation an equal default tile inside every Workspace.

The strongest product model is:

Surface	Primary job
PacketCode	Best terminal-native coding agent and automation-friendly inner loop
PacketADE Workspace	High-density professional workroom for PacketCode, Claude Code, Codex CLI, Gemini, OpenCode, shells, files, and Git
PacketADE Agents	Agent-first command center for API/subscription conversations, approvals, plans, visual diffs, and cross-project attention
Flight Deck	Structured delivery: plans, tasks, attempts, reviewers, integration, and publish decisions
Monitor	Read-only operational visibility on another display
PacketAgent	Durable bounded execution after PacketADE closes

This is the pro-user version of the BridgeSpace idea: keep the excellent CLI workroom, but do not force CLI operation, asynchronous agent supervision, and structured Flight delivery into one spatial grammar.

The recommendation is a staged Option B:

1. Reintroduce a first-class **Agents** surface in the existing main window, backed by the current conversation engine.
2. Make new Workspaces CLI/PTY-first and put PacketCode first when it is installed.
3. Preserve existing conversation panes and an explicit **Open in Workspace** compatibility path during the transition.
4. Add a detachable native Agents window only after agent state has an explicit single-writer/backend-authoritative contract.
5. Decide whether to remove new GUI-agent attachment from Workspaces only after dogfood evidence. There is no reason to delete compatibility for existing attached conversations.

The Settings surface should remain, but its current 16-section information architecture should not. It mixes preferences, credentials, operational tools, status, and navigation. Before adding more switches, fix the controls that do not govern runtime behavior and reorganize Settings into six coherent groups.

The decision in one table

Option	Product clarity	Pro workflow	Near-term engineering risk	Decision
A. Keep the current mixed Workspace	Medium-low	Good for two to four adjacent actors	Low	Do not choose as the north star
B. CLI-first Workspace plus Agents surface	High	Best balance across foreground CLI and delegated work	Medium and controllable	Recommended
C. Interactive native Agent window now	High visually	Excellent for multiple monitors	High with current frontend ownership	Target later, not first
D. Delete PacketADE GUI agents	Low	Throws away provider, review, and approval advantages	High product loss	Reject

Why the pro user still uses agents

A professional user does not choose between "CLI" and "agents" once for the whole product. The same person changes modes throughout the day:

- use PacketCode or another CLI for the fast, interactive, high-agency loop;
- delegate exploratory, reviewable, or longer tasks to GUI/API agents;
- supervise structured multi-agent delivery in Flight Deck;
- hand durable work to PacketAgent when the ADE should be allowed to close.

PacketCode strengthens the case for a CLI-first Workspace. It does not make PacketADE's GUI-agent engine redundant. The GUI engine owns capabilities a TUI should not have to reproduce:

- eight provider/auth identities with live status;
- subscription and API-key transports behind one event contract;
- permission and edit review;
- plans, tool cards, diffs, worktree endings, and cost;
- resumable local/SSH conversations;
- MCP trust snapshots and Memory context;
- global attention across projects.

The product mistake would be making either mode swallow the other.

Facts from the current PacketADE code

Workspaces are already a substantial CLI product

The Workspace supports persistent PTY panes, local and SSH projects, mosaic layouts, focus/zoom, model and effort overrides, prompt templates, pinned commands, a lightweight editor, and local/remote Git operations. Its client set includes PacketCode, Claude Code, Codex CLI, Gemini, OpenCode, and a plain terminal. Named templates already cover Solo, Duo, Review Trio, Research, and Full Stack shapes.

Relevant implementation:

- `src/components/views/WorkspaceView.tsx`
- `src/components/workspace/WorkspacePane.tsx`
- `src/components/workspace/WorkspaceCreationModal.tsx`
- `src/components/workspace/GitDashboard.tsx`
- `src/stores/workspaceStore.ts`

A conversation tile is also a full application

AgentChatPane and ConversationTile contain streaming transcript/tool events, planning, Memory context, permission and edit approvals, review/diff UI, mode/model controls, queued sends, retry/fork, attachments, tokens, cost, and SSH execution context. These interactions want reading width and stable review space. The tile already auto-zooms for review because ordinary mosaic width is not a natural review surface.

Relevant implementation:

- `src/components/agents/AgentChatPane.tsx`
- `src/components/workspace/ConversationTile.tsx`

- src/stores/agentTaskStore.ts
- src/stores/agentApprovalStore.ts
- src/stores/reviewStore.ts

The conversation engine is not intrinsically owned by a Workspace

AgentConversation has no workspaceId. Placement points in the other direction: a WorkspacePane may reference a conversationId. Flight planning, reviewer gates, prompt paths, and other flows already create headless conversations before any Workspace placement.

This is the most important implementation fact. The expensive runtime, event, approval, persistence, Memory, MCP, review, and worktree work can remain. Option B primarily changes navigation and presentation.

The current mixed surface carries real product tax

To combine both models, PacketADE needs:

- conversation pane kinds and downgrade-compatible carriers;
- transient draft leaves;
- wrapper workspaces for unplaced conversations;
- conversation-to-pane garbage collection;
- reconciliation after old-binary saves;
- unified Fleet projection and archive fan-out;
- special focus, auto-zoom, status, and hidden-mount behavior.

This code is well tested, but the complexity exists because two different interaction models were forced into one mosaic.

There is a current focus/selection seam

Conversation tile focus updates the active Workspace pane, while the right inspector follows `selectedConversationId`. Focusing a tile does not consistently select the same conversation. This is fixable, but it shows that Workspace focus and agent selection are still separate concepts below the "single surface" promise.

A blunt rollback would be irresponsible

The July conversation-tile program touched roughly 137 files and added about 14,000 lines including tests and migration scaffolding. That is not a reason to keep a weak product default, but it is a reason to preserve:

- the shared conversation engine and UI components;
- focused keyboard/approval safety;
- review and worktree endings;
- pane/read compatibility for persisted layouts;
- headless Flight and reviewer behavior.

Change the placement model; do not delete the capability model.

What the current market says

The strongest current pattern is not "embedded or separate." It is **one session model with multiple purpose-built presentations**.

Cursor

Cursor 3 introduced an agent-centered, multi-workspace Agents Window while retaining the ability to switch to the IDE. Cursor 3.1 added a persistent tiled layout for supervising and comparing several agents. The agent-first surface still includes files, diffs, browser, commit, and PR workflow.

Inference for PacketADE: local foreground work may remain close to the Workspace, while high-concurrency and cross-project supervision deserves an agent-first surface.

Visual Studio Code

Microsoft now documents the split explicitly:

- Chat view is code-first and stays with editor/debug/task context.
- Agents Window is agent-first and spans projects.
- Both surfaces share the same sessions and settings.
- A chat may live in a sidebar, editor tab, or separate window.

This is the cleanest validation of a hybrid PacketADE design. PacketADE's foreground anchor should be PTY/CLI rather than a full IDE.

OpenAI Codex

The Codex app is a separate command center for parallel agents, worktrees, and review. It consumes history and configuration from Codex CLI and IDE surfaces. A separate agent application therefore does not weaken a CLI when identity, configuration, and handoff remain shared.

Claude Code Desktop

Claude Desktop provides an agent-first pane workspace for chat, diff, terminal, file, plan, task, and subagent views. Anthropic positions Desktop for parallel supervision and visual review, and CLI for scripting, automation, or terminal preference.

This maps directly to PacketADE plus PacketCode. PacketADE can eventually do better by preserving one conversation identity across presentations.

BridgeMind BridgeSpace

BridgeSpace is the clearest one-window counterexample: a terminal-grid-first workroom around external coding CLIs. That validates the CLI-first Workspace direction. Even BridgeSpace later added detached OS windows for editor, Git, browser, and other panels, showing the practical pressure created by one-window density.

Market conclusion

The market has converged on three layers:

1. a code/CLI-first foreground surface;
2. an agent-first supervision surface;
3. asynchronous or remote execution that can outlive the foreground.

PacketADE already has the primitives for all three. Its opportunity is to give them clear jobs and stronger handoffs instead of copying an IDE chat sidebar.

Recommended Workspace and Agents contract

Workspace

- Terminal/CLI panes only by default.

- PacketCode is recommended first when detected.
- Rename "Add Agent" to **Add CLI / Add Session**.
- Improve saved layouts, broadcast commands/prompts, reconnect/replay, health, runbooks, and pane-level Git/project status.
- Keep small status/attention affordances, not full agent conversations, in the default mosaic.

Agents

- Cross-project conversation and attention list.
- One primary agent conversation, with optional side-by-side comparison later.
- Existing inspector, approval, plan, Memory, review, diff, and worktree-ending components.
- Launch by provider, auth identity, model, profile, project, local/SSH target, permission mode, and worktree isolation.
- Direct handoffs: **Open project in Workspace**, **Send/continue in PacketCode**, **Attach terminal**, **Open Git ending**, **Add to Flight**.

Presentation rules

- Moving a conversation changes presentation, not ownership.
- Conversation ID, provider process, worktree, context, approvals, and history stay unchanged.
- Review and Git have one authoritative state; they are not cloned per view.
- Existing persisted conversation panes remain readable during migration.

Why the native Agent window should be phase two

The market case for a native window is strong, especially for multiple monitors. The current PacketADE frontend is not ready for two interactive writers:

- Zustand stores and listener cleanup maps are WebView-local.
- `agentTaskStore` installs scoped Tauri listeners per active conversation.
- approval queues, drafts, review state, streaming buffers, and selections live in frontend stores;
- both windows could hydrate and save the same conversation data;
- duplicate listeners could process the same event or race persistence.

The current Monitor window is deliberately read-only; it does not prove safe multi-writer interaction.

Phase-two native interactivity should use one of these contracts:

1. the main window owns runtime and persistence; popouts are projections whose commands route through the owner; or
2. Rust owns canonical conversation, approval, and revisioned persistence state while all windows subscribe to versioned snapshots/events.

Until then, restore the Agents route in the same main window. That proves the information architecture without creating a state-integrity project inside a surface redesign.

Low-risk rollout

Phase 1 - prove the split

- Restore an `agents` route with conversation fleet, primary `AgentChatPane`, and inspector.
- Retarget conversation deep links to Agents.
- Keep existing Workspace conversation panes compatible.

- Fix the focus/selection seam in whichever view owns a conversation.

Phase 2 - change defaults

- Make Workspace creation CLI-only by default.
- Move GUI-agent creation to Agents.
- Put PacketCode first in Workspace.
- Add explicit cross-surface and PacketCode handoffs.
- Stop creating wrapper workspaces for new conversation navigation.

Phase 3 - compatibility migration

- Detach old conversation pane placement without deleting conversation files, transcripts, active sessions, approvals, worktrees, or Flight links.
- Retire draft/wrapper/tile-only machinery after at least one compatibility release and full migration proof.

Phase 4 - detachable Agents window

- First ship a read-only or single-owner projection.
- Make it interactive only after a single-writer contract and multi-window approval/persistence tests.

Phase 5 - decide from dogfood evidence

Track:

- sessions started in Agents versus attached to Workspace;
- attach/open-in-other-surface frequency;
- simultaneous visible conversations;
- time from `needs_you` to response;
- single-project foreground versus cross-project background work;
- single-monitor versus multi-monitor usage.

Only then decide whether new GUI-agent attachment should disappear completely.

Settings audit: bottom line

Keep Settings, but make it authoritative. Every control must answer:

1. What does this affect?
2. At what scope?
3. When does it take effect?
4. Did the runtime accept it?

The current sidebar still has 16 root sections. It mixes durable preferences, credentials/integrations, operational management, status, and navigation. Several controls persist desired state without changing effective runtime state.

Settings findings that should block more polish

P0 - MCP provider scope and allowed-tool controls are not enforced

The PacketADE MCP provider UI persists Project/Global scope and individual tool checkboxes, but `mcp_server_start` receives only port and `allowWrites`. Rust constructs the full static tool router.

Until fixed, disable/remove the scope and tool controls and describe the actual loopback endpoint plus its real write gate. The complete fix is a frozen backend policy that filters `list_tools`, `call_tool`, and resource reads and returns the effective policy to the UI.

P0 - password SSH configuration cannot save a password

The Server form offers Password auth but has no password field. Rust can read or check an SSH keyring value, but no active command writes/deletes one. Users are told to re-save a server even though Settings cannot save the secret.

Either hide Password auth or implement secure set/delete commands, a secret field with stored state, and a Test action that verifies host key, auth, and base path. Never persist the password in frontend state or the server DTO.

P0 - AI Provider Routing is unconsumed

The routing tab says tasks auto-fill from its defaults, but its resolver has no production call site. It also selects CLI agent configs rather than the eight API provider rows.

Remove it until consumed, or wire it end to end and rename it **Task Role Defaults**. Launch review must show the resolved agent/model and source.

P1 - two Agent settings are placebo

Default launch location and Start right rail collapsed persist but have no production consumer. The draft path hardcodes local launch, and rail collapse is not read by layout.

Remove or wire them. Surface real hidden preferences such as transcript density and worktree cleanup instead.

P1 - MCP defaults are unsafe when names collide across scopes

Global and project servers with the same name render with scoped IDs, but agent defaults and trust filtering persist/filter by name only. Migrate selections to stable scoped IDs and show override precedence.

P1 - remote Workspace Project information can be wrong

The Project card displays the global local `layoutStore.projectPath`. For an SSH Workspace that value intentionally remains the previous local path.

Project management belongs under Workspaces & Terminal and must read the active Workspace's local/SSH identity directly.

P1 - Settings cannot deep link to a section

`activeSection` is local component state. Calls such as "Manage in Settings > API Keys" can open only the broad Settings view and land on General.

Add an `openSettings(section, anchor?)` contract and persist the last section while allowing explicit links to win.

P1 - Flight settings can say Saved before persistence succeeds

Orchestration settings update memory, fire an unawaited backend merge/write, and swallow errors. The card immediately displays Saved. Trailer changes can also create overlapping read/merge/write operations.

Make saves awaitable, revisioned, and error-visible. YOLO policy is a safety boundary and cannot be best-effort UI state.

Other important Settings corrections

- Rename **GitHub** to **Git Hosts** because the card also owns Gitea/Forgejo.
- Rename PacketAgent **Test connection** to **Test endpoint health**; W9 health does not verify bearer token or Workspace identity.
- Add confirmation and dependency impact before deleting SSH servers.
- Validate Agent Profile model/tool values against provider capability.

- Add provider key validity/reachability tests, not only key existence.
- Make notification permission denial visible.
- Validate Issue ticket prefixes and support taxonomy rename/merge/delete.
- Make MCP copy transport-agnostic rather than Claude-only.
- Move hardcoded Release Trust status into live Diagnostics/About.
- Stop saying no crash reports proves the app is healthy.
- Put Project Rules near the active Workspace; it is an operational editor, not a global preference.
- Combine Dictation and its shortcuts, fix API Keys deep links, and add model/disk reset controls.

Recommended six-group Settings IA

1. General

- Appearance: system/dark/light.
- Notifications and OS permission state.
- Global keyboard shortcuts.
- Startup/reopen behavior if adopted.

2. Workspaces & Terminal

- Project and Workspace defaults.
- Terminal font, size, line height, cursor, scrollbar, copy behavior, and renderer.
- Default shell/profile and per-Workspace environment.
- Workspace layout/template and restore defaults.
- CLI Clients, with PacketCode first-party controls.
- Remote Hosts, secrets, test, dependencies, and guarded delete.
- External editor preference and CLI doctor.

3. Agents & Models

- Accounts and credentials grouped by vendor.
- Provider endpoints, connection tests, and model refresh.
- Default provider/model/mode/profile/target.
- Transcript density, archive, failover, and worktree behavior.
- Provider-aware profiles and non-secret import/export.

4. Automation

- Flight commit attribution.
- Bounded autonomy and effective policy review.
- Worktree cleanup and cost limits.
- Task Role Defaults only after runtime wiring.
- PacketAgent endpoint, identity, and handoff status.

5. Integrations & Data

- Git Hosts and PR defaults.
- MCP client/provider and effective trust policy.
- Issues taxonomy and mirror summary.
- Memory settings, location, privacy, export/delete.
- Dictation and first-party modules.

6. Security & Diagnostics

- Credential inventory without secret disclosure.
- Trust and provenance.
- Sidecar, CLI, SSH, provider, schema, and runtime diagnostics.
- Crash reports and diagnostic bundle export.
- Build channel, version, signing, and updater state.

Every row/card should show an **App**, **Project**, **Workspace**, **New conversations**, or **New Flights** scope badge.

Settings implementation order

Cleanup

1. Hide AI Routing.
2. Remove or wire unused Agent controls.
3. Disable placebo MCP provider controls.
4. Hide or complete password SSH.
5. Guard server deletion.
6. Add Settings deep links.
7. Fix remote Project identity and stale copy.
8. Correct PacketAgent health wording.

Reliability

1. Make backend-backed saves awaitable and revisioned.
2. Use scoped MCP server IDs.
3. Add tests that assert runtime consumption, not only persistence.
4. Add integration tests for SSH secret lifecycle, effective MCP policy, orchestration save failure, and Settings deep links.
5. Add provider/profile/ticket-prefix validation.

Product quality

1. Implement the six-group IA with search and scope badges.
2. Add terminal and CLI preferences.
3. Add provider/CLI/SSH diagnostics.
4. Add non-secret export/import and data management.
5. Document inheritance: app default -> project/Workspace override -> conversation/Flight snapshot.

Decisions requested from the owner

This report recommends, but does not silently approve, the following product changes:

1. Approve CLI-first Workspaces as the north star.
2. Approve a same-main-window Agents surface as the first implementation.
3. Preserve optional/read-compatible Workspace agent attachment during migration.
4. Defer interactive native Agent windows until single-writer state exists.
5. Approve the six-group Settings IA.
6. Approve immediate removal/disablement of placebo settings before adding new terminal preferences.

Evidence limits

High confidence:

- current PacketADE source behavior and coupling;
- current first-party product layouts;
- the runtime gaps identified by missing production consumers;
- the risk of two interactive WebViews owning current frontend state.

Medium confidence:

- the exact split the owner will prefer after daily dogfood;
- how often PacketCode will replace GUI-agent interaction;
- whether a permanent optional attachment path earns its maintenance cost.

Not established:

- a neutral cross-product user study proving separate windows improve productivity;
- PacketADE-specific telemetry for session placement;
- live validation of the Settings controls against every configured external account/server.

The staged recommendation exists to collect the missing evidence without destroying shipped work.

Primary sources

Current first-party web sources, checked 2026-07-29:

1. Cursor, [Meet the new Cursor](#), 2026-04-02.
2. Cursor, [Cursor 3.1 tiled Agents Window](#), 2026-04-13.
3. Microsoft, [Use the Chat view](#).
4. Microsoft, [Use the Agents window](#).
5. Microsoft, [Agent sessions and where agents run](#).
6. OpenAI, [Introducing the Codex app](#), 2026-02-02, Windows update 2026-03-04.
7. Anthropic, [Claude Code on desktop](#).
8. BridgeMind, [BridgeSpace](#).
9. BridgeMind, [Changelog](#).

Repository evidence:

- [conversation-tile-design.md](#)
- [tile-program/plan.md](#)
- [bridgemind/bridgespace-competitive-brief.md](#)
- src/App.tsx
- src/stores/appStore.ts
- src/stores/workspaceStore.ts
- src/stores/agentTaskStore.ts
- src/components/views/WorkspaceView.tsx
- src/components/views/ToolsView.tsx
- src/components/workspace/ConversationTile.tsx
- src/components/workspace/FleetSidebar.tsx
- src/components/agents/AgentChatPane.tsx
- src/stores/mcpProviderStore.ts
- src/stores/orchestrationSettingsStore.ts
- src-tauri/src/mcp_server/mod.rs
- src-tauri/src/commands/ssh_keys.rs